

Introduction to C Programming

1. What is C?

- C is a general-purpose programming language used to build software that needs speed, control and efficient use of system resources.
- It is widely used for operating systems, embedded systems, device drivers, compilers, networking software and performance-critical applications.
- C gives the programmer direct control over memory and low-level operations while still providing structured programming constructs.
- C is standardized, portable at the source-code level when programs avoid platform-specific features, and available on almost every major computing platform.

Think of C this way: C sits fairly close to the hardware, but it still lets us write programs using readable constructs such as variables, functions, conditions and loops.

2. Why is C still important?

Feature	What it means for a programmer
Performance	Programs can be compiled to efficient native machine code.
Memory control	Pointers and dynamic memory allow precise control over memory.
Small runtime overhead	C has a small language/runtime footprint compared with many higher-level environments.
Portability	The same C source can often be compiled for different platforms with limited changes.
Foundation	Many languages, libraries and systems concepts are easier to understand after learning C.
Predictable low-level behavior	You can see more clearly how data types, bytes, addresses and memory relate.

3. C compared with other popular languages

Aspect	C	Python	Java
Typical execution	Compiled to native machine code	Usually executed by a language runtime; CPython compiles source to bytecode first	Compiled to bytecode and executed by the JVM using interpretation and/or JIT compilation
Memory management	Mostly manual / programmer controlled	Automatic memory management	Automatic garbage collection
Abstraction level	Closer to hardware	Higher level	Higher level than C
Typical strength	Performance, systems, embedded	Rapid development, scripting, data/AI	Portability, large applications, ecosystem
Learning value	Builds strong fundamentals in memory, types and execution	Fast to start and highly expressive	Strong object-oriented and runtime concepts

Important: These are typical characteristics, not rigid rules. A language and its implementation are not the same thing. For example, Python also has a compilation-to-bytecode step, and Java uses JIT compilation in modern JVMs.

4. Basic C Program - Boilerplate Code

```
#include <stdio.h>

int main(void)
{
    printf("Hello, C!\n");
    return 0;
}
```

Line	What it does	Why it is written
#include <stdio.h>	Includes declarations for standard input/output functions such as printf and scanf.	Without the proper declaration, the compiler may not have the information needed for these functions.
int main(void)	Defines the main function where program execution begins.	A C program needs a main function as its entry point.
{	Starts the body of main.	Groups the statements belonging to the function.
printf(...)	Sends formatted text to standard output.	Used to display information to the user.
return 0;	Ends main and returns status 0 to the operating system.	Conventionally indicates successful completion.
}	Ends the function body.	Marks the end of main.

5. Syntax vs Semantics

Term	Simple meaning	Example
Syntax	Rules for writing valid C code.	Missing semicolon or unmatched braces can cause a syntax/compile error.
Semantics	What the valid code actually means or does.	$x = y + 2$; is syntactically valid, but using an uninitialized y is a programming problem.

```
int x = 10
printf("%d", x);
```

Remember: Syntax is about structure; semantics is about meaning/behavior.

6. Variables, Constants and Identifiers

- Variable: a named memory location whose stored value can change during program execution.
- Constant/literal: a fixed value written directly in the program, such as 10, 3.14 or 'A'.

- Identifier: a name given to a variable, function or other program element.
- Choose meaningful names: totalMarks is easier to understand than tm.

Good practice	Example
Meaningful name	float temperature;
Consistent naming	int totalMarks;
Avoid ambiguous names	int x; is okay for a temporary calculation, but not ideal for business logic.
Avoid keywords	int return; is invalid because return is a C keyword.

7. Primitive Data Types

Type	Typical purpose	Example	Common printf
char	Single character / small integer representation	char grade = 'A';	%c
int	Whole numbers	int age = 20;	%d
float	Decimal values with single precision	float price = 19.5f;	%f
double	Decimal values with higher precision than float	double pi = 3.1415926535;	%f
void	Represents no value / no return value in common function uses	void display(void);	-

Do not memorize only sizes: The exact size of integer types is implementation-dependent. Use sizeof when you need the actual size on the current system.

```
printf("%zu\n", sizeof(int));
```

8. printf() - Displaying Output

```
printf("format string", values);
```

Specifier	Typical use	Example
%d	int	printf("Age = %d", age);
%u	unsigned int	printf("%u", count);
%f	float/double output	printf("%.2f", price);
%c	char	printf("Grade = %c", grade);
%s	C string (char array)	printf("Name = %s", name);
%x	Hexadecimal integer	printf("%x", 255);
%%	Print a percent sign	printf("50%%");

Useful formatting	Meaning	Example output
%.2f	Show 2 digits after the decimal point	3.14
%8d	Minimum field width of 8 characters	42
%-8d	Left-align within width 8	42
%08d	Pad with zeros to width 8	00000042

9. scanf() - Taking Input

```
scanf("format string", addresses);
```

- For an ordinary scalar variable, scanf usually needs the variable address, written using &.
- scanf returns the number of input conversions that succeeded. A good program checks this return value when input may be invalid.
- For a character array read using %s, do not write &name. The array name already acts as a pointer to its first element in this context.

Data	scanf format	Correct argument
int	%d	&n
float	%f	&x
double	%lf	&x
char	%c	&ch
word string char[]	%s	name

float vs double - very common viva question: With printf, both float and double are normally printed using %f because float arguments are promoted to double in a variadic call. With scanf, %f reads float and %lf reads double.

10. Operators in C

Category	Operators	Use
Arithmetic	+ - * / %	Mathematical calculations
Relational	< > <= >= == !=	Compare two values; result is 0 or 1
Logical	&& !	Combine or negate conditions
Assignment	= += -= *= /= %=	Store/update values
Unary arithmetic	+ -	Indicate sign / value
Conditional	?:	Compact selection between two expressions
Bitwise	& ^ ~ << >>	Operate on individual bits

Scope for these notes: For beginner practice, focus first on arithmetic, relational, logical and assignment operators. Increment/decrement are intentionally treated separately later.

11. Arithmetic Operators - Understand the Result

```
int a = 17, b = 5;
printf("%d\n", a + b); // 22
printf("%d\n", a - b); // 12
printf("%d\n", a * b); // 85
printf("%d\n", a / b); // 3
printf("%d\n", a % b); // 2
```

- When both operands are int, / performs integer division. 17 / 5 gives 3, not 3.4.
- % gives the remainder. 17 % 5 gives 2.
- If you need a decimal result, make at least one operand floating point: 17 / 5.0 gives 3.4.

```
printf("%.1f\n", 17 / 5.0); // 3.4
```

12. Operator Precedence - Which Operation Happens First?

Priority idea	Operators	Example
Higher	* / %	a + b * c -> multiplication first
Next	+ -	a * b + c - d
Comparisons	< > <= >=	a + b > c
Equality	== !=	a > b == 1
Logical AND	&&	condition1 && condition2
Logical OR		condition1 condition2
Assignment	= += -= ...	x = a + b

```
int result = 10 + 4 * 2 - 6 / 3;
printf("%d", result);
// 16
```

Best habit: When an expression is even slightly hard to read, use parentheses. Clear code is better than relying on memory of precedence rules.

13. Relational and Logical Operators

```
int age = 20;
printf("%d\n", age >= 18);    // 1
printf("%d\n", age < 18);     // 0
printf("%d\n", age >= 18 && age <= 60); // 1
```

- In C, a relational or logical expression evaluates to 0 for false and 1 for true in the usual integer context.
- Do not confuse = (assignment) with == (equality comparison).

```
int x = 10;
if (x == 10) {
    printf("Equal");
}
```

14. Assignment Operators

Operator	Equivalent idea	Example
=	Assign	x = 10;
+=	Add then assign	x += 5; // x = x + 5
-=	Subtract then assign	x -= 5;
*=	Multiply then assign	x *= 5;
/=	Divide then assign	x /= 5;
%=	Remainder then assign	x %= 5;

15. Increment and Decrement - A Common Source of Confusion

- ++ increases a variable by 1; -- decreases a variable by 1.
- Prefix form changes the value before the expression uses it; postfix form uses the old value first and changes it afterward.

- For beginner code, avoid writing complicated expressions that mix ++/-- with other operators. Separate the statements for readability.

```
int a = 5;
int x = ++a; // a becomes 6, x becomes 6

int b = 5;
int y = b++; // y becomes 5, b becomes 6
```

Good programming habit: Prefer simple, separate statements over clever one-line expressions when teaching or maintaining code.

16. Bitwise Operators - Know the Idea

Operator	Meaning	Example
&	Bitwise AND	5 & 3 = 1
	Bitwise OR	5 3 = 7
^	Bitwise XOR	5 ^ 3 = 6
~	Bitwise NOT	Flips bits
<<	Left shift	3 << 1 = 6
>>	Right shift	8 >> 1 = 4

- Bitwise operators work on the binary representation of integer values.
- They are common in embedded systems, device drivers, flags and low-level programming.

17. From a .c File to a .exe - What Really Happens?

```
first.c
|
v
Preprocessor
|
v
Expanded source
|
v
Compiler
|
v
Assembly code
|
v
Assembler
|
v
Object file (.o / .obj)
|
v
Linker + libraries
|
```

v

Executable (.exe on Windows)

Stage	What happens	Typical issue caught here
Preprocessing	Handles #include, #define and conditional compilation directives.	Missing header or malformed preprocessor directive
Compilation	Translates C source into lower-level code such as assembly and checks language rules/types.	Syntax errors, many type/format warnings/errors
Assembly	Converts assembly code into machine-code object instructions.	Assembler errors
Linking	Combines object files and required libraries into an executable.	Undefined reference / unresolved external symbol
Execution	OS loads the executable and the CPU runs its machine instructions.	Runtime errors, wrong logic, crashes

Simple memory line: `.c -> preprocess -> compile -> assemble -> link -> executable -> run`

18. What GCC Is Doing for You

`gcc first.c -o first`

- gcc invokes the required compilation stages and produces the executable named first on many systems.
- On Windows, the executable is commonly first.exe depending on the toolchain.
- Useful development command: `gcc -Wall -Wextra first.c -o first`. Warnings often reveal format-string mistakes and other bugs before execution.

Very useful habit: Treat compiler warnings as feedback, not as noise. Clean, warning-free code is a strong goal.

19. Do and Do Not

Do	Do not
Use meaningful variable names.	Use names like a1, a2, a3 everywhere without context.
Initialize variables before using their values.	Assume a newly declared local variable already contains 0.
Match format specifiers to the actual types.	Mix %d, %f, %lf, %c etc. carelessly.
Use & with ordinary scalar variables in scanf.	Write <code>scanf("%d", n)</code> for an ordinary int variable.
Use a width for %s when reading into a fixed-size char array.	Use unlimited %s input into a small array.
Use parentheses when an expression may be confusing.	Rely on complicated precedence rules to save characters.
Compile with warnings enabled.	Ignore warnings repeatedly.
Break a complicated calculation into readable steps.	Try to write a clever one-line expression that is hard to verify.

20. Quick Debugging Checklist

- Compiler error? Read the first error carefully; later errors may be side effects.
- Wrong output? Check data types, integer division and operator precedence.

- printf looks wrong? Verify every format specifier against the value being passed.
- scanf behaves oddly? Check & usage, input format and its return value.
- Character appears to be skipped? Check whitespace before %c and whether a previous input left a newline.
- Program crashes? Check array bounds, pointers and invalid memory access.
- Compiler warnings? Fix them before assuming the program is correct.

21. Small Practice Examples

```
/* Example 1: read and display values */
int age;
double height;
char grade;

printf("Enter age, height and grade: ");
if (scanf("%d %lf %c", &age, &height, &grade) == 3) {
    printf("Age = %d\n", age);
    printf("Height = %.2f\n", height);
    printf("Grade = %c\n", grade);
}
```

Notice: For interactive character input, scanf(" %c", &grade) is usually safer because the leading space skips leftover whitespace.

```
/* Example 2: arithmetic and formatted output */
int a = 17, b = 5;
printf("Sum = %d\n", a + b);
printf("Quotient = %d\n", a / b);
printf("Remainder = %d\n", a % b);
printf("Decimal quotient = %.2f\n", a / 5.0);
```

22. Quick Revision - What a Good C Programmer Should Know First

Concept	Remember this
Program structure	#include <stdio.h>, main(), statements, return 0;
Syntax	Rules for writing valid C code.
Semantics	Meaning/behavior of valid code.
Variables	Named storage for data; declare before use.
Data types	Choose a type that matches the kind of value you need.
printf	Format string + values; match every specifier correctly.
scanf	Format string + addresses for ordinary variables; check return value.
Arithmetic	Know +, -, *, /, % and integer division.
Comparison	== compares; = assigns.
Logic	&& means AND, means OR, ! means NOT.
Precedence	*, /, % happen before + and -; use parentheses for clarity.
Compilation flow	.c -> preprocess -> compile -> assemble -> link -> executable.
Professional habit	Compile with warnings, keep code readable, and fix warnings.

